

MQTT:

MQTT - a lightweight, low bandwidth message queuing transport where it uses pub/sub to deliver the messages

publisher - the client that sends a msg to a topic

subscriber - the client that receives a msg from a topic

broker - a server where it sends/receives the msgs from/to the clients, nobody talks directly, the broker stands in the middle

topic - the channel name the msgs travel on, pub and sub must point at the same one

what i did in this track in short

- ran and secured a mosquitto broker on windows (service cmds, conf file, connection limits)
- built the full TLS/mTLS certificate chain with openssl (CA, server, client) and wired it into mosquitto.conf
- wrote a paho python client ([mqttdemo.py](#)) that does the complete pub/sub round trip over TLS
- put the whole concept to work in the Machine_Sensor_API capstone (see below)

broker basics - how i ran mosquitto on windows

- where it deals with starting/stopping/restarting the service
the common cmds are:
 - net start mosquitto - starts the broker as a windows service
 - mosquitto -c mosquitto.conf - runs it in the foreground with my conf file (used when testing conf changes)
 - net stop mosquitto - stops the service (or Ctrl + c for the foreground run)
 - net stop mosquitto && net start mosquitto - the restart, needed after every conf edit
- default install files - mosquitto.conf and the certs live in C:/Program Files/Mosquitto/, edit the conf as admin and save, never generate it with echo (it erases the new lines, do the changes manually)
- max_connections - total connections allowed per listener (one port)
- global_max_connections - total connections allowed per broker (all listeners together)
- global_max_clients - how many devices the broker can remember
- connection limits are set in the conf file so we can restrict how many devices connect through that port/broker

TLS/mTLS - the full certificate chain i built with openssl

- why - plain mqtt sends everything readable on the wire, TLS encrypts it and mTLS makes both sides prove who they are
- what each file is for
 - ca.key - the CA private key, created encrypted with -des3 (self-signed root)
 - ca.crt - the CA certificate, signs both server and client certs and verifies both sides
 - server.key / server.csr / server.crt - the broker identity, the CSR carries CN=ip_or_domain
 - client.key / client.csr / client.crt - the client identity, the CSR carries CN=client1the cmds in order are
 - openssl genrsa -des3 -out ca.key 2048 - creates the encrypted CA key
 - openssl req -new -x509 -out ca.crt -key ca.key -days 365 - creates the CA cert from that key
 - openssl genrsa -out server.key 2048 - server key for the broker
 - openssl req -new -out server.csr -key server.key -subj "/CN=domain" - server CSR, the -subj flag skips the interactive prompts
 - openssl x509 -req -out server.crt -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -days 365 -extfile san.cnf -extensions v3_req - signs the server cert with the CA, -CAcreateserial makes the .srl file openssl uses to track our CA, SAN comes from san.cnf
 - openssl genrsa -out client.key 2048 - client key
 - openssl req -new -out client.csr -key client.key -subj "/CN=client1" - client CSR, with use_identity_as_username=true the CN becomes the username (client1)
 - openssl x509 -req -out client.crt -in client.csr -CA ca.crt -CAkey ca.key -CAcreateserial -days 360 - signs the client cert
- the -subj flag - works for BOTH server and client CSRs, format "/C=IN/ST=State/L=City/O=Org/CN=identity" (CN is must at least), essential for scripting certs for multiple clients
- when the ip changes (different wifi router) - no need to regenerate everything, just add the new ip in san.cnf and regenerate server.crt, that is why the SAN entry saves you

SAN - subject alternative name (san.cnf)

- what - an extra field in the server certificate listing every ip/domain the server accepts connections on

- openssl 3.x REQUIRES it - without SAN the certificate verify simply fails
my san.cnf is
 - [v3_req] - the section for X.509 v3 extensions
 - subjectAltName = @alt_names - the field, pointing at the [alt_names] section
 - IP.1 / IP.2 - the valid ip addresses (10.58.166.132 and 127.0.0.1)
 - DNS.1 = localhost - the valid domain name
- rule i learned - with an ip in the SAN a new ip only needs a san.cnf edit + server.crt regen, with a domain/localhost it is the same idea, either way the CA and client certs never change

mosquitto.conf - my real TLS listener

- where - C:/Program Files/Mosquitto/mosquitto.conf, edited as admin, certs stored in the same install folder
my conf is
 - listener 8883 - the TLS port (8883 is the default for mqtt over TLS)
 - cafile - the CA cert, verifies both server and client
 - certfile - the server cert, proves the broker identity
 - keyfile - the server private key for the handshake
 - require_certificate true - forces every client to present a cert (mTLS)
 - use_identity_as_username true - takes the CN from the client cert as the username
 - allow_anonymous false - rejects clients without a valid cert
 - password_file - NOT needed when require_certificate is on, the cert is the identity
- server side and client side must agree - the conf paths on the server, the tls_set paths on the python client, and the same port 8883 on both, or you get an SSLCertVerificationError
- ca.crt also has to be placed in the windows Trusted Root Certification Authorities store so the system trusts our CA

testing through the terminal

- mosquitto_sub --cafile ca.crt --cert client.crt --key client.key -h 10.58.166.132 -p 8883 -t test - subscriber with the full cert trio
- mosquitto_pub --cafile ca.crt --cert client.crt --key client.key -h 10.58.166.132 -p 8883 -t test -m "hello there" - publisher with the same trio
- always send msgs through --cafile/--cert/--key, a plain connect is refused by the listener

python client - paho ([mqttdemo.py](https://github.com/eclipse/paho.mqtt.python))

- till paho 2.x the callback api defaulted to version 1, since the 2.0 release we have to assign it ourselves to version 2 explicitly, otherwise we silently get the legacy api
- one MQTTClient class where **init** sets everything up (broker, port, topic, msg, client_id, tls certs, callbacks) so the methods stay small
- paths in two sides - server side in mosquitto.conf (the service starts before the python code) and client side in the .py file (we run and handle all the pub/subs there)
- the demo choreography - on_connect() calls subscribe() and on_subscribe() calls publish(), so all five callbacks fire in one run and the subscriber gets the ack from the server before the publish, it is a demo flow on purpose, a real IIoT client would subscribe to device topics and process whatever arrives

the five callbacks - predefined functions we register, paho runs them automatically

- on_connect(client, userdata, flags, reason_code, properties) - runs whenever th
 - on_subscribe(client, userdata, mid, reason_codes, properties) - ack from the se
 - on_publish(client, userdata, mid, reason_code, properties) - ack that the publi
 - on_message(client, userdata, message) - runs whenever a msg arrives, decodes me
 - on_disconnect(client, userdata, disconnect_flags, reason_code, properties) - ru
-
- every callback is wrapped in its own try/except with logging.exception so one bad callback never kills the loop silently

loops - how the mqtt communication runs in the background

- loop_forever() - blocks the main thread, keeps the connection alive and reconnects automatically, we use it when we only do pub/sub
- loop_start() - runs the loop on a background thread, we use it when we manage multiple clients
- connect() opens the tcp connection, the loop then keeps it alive while the callbacks do their work

return codes vs callbacks - both checks matter

- the immediate return answers "did my request go out" - `subscribe()` gives back `(res, mid)` and `publish()` gives back an object with `.rc`, both checked against `mqtt.MQTT_ERR_SUCCESS`
- the callbacks answer "what did the broker actually do" - that is why `on_connect` checks `reason_code.is_failure` before subscribing

error types happened to encounter

- `ConnectionRefusedError` - broker not running/wrong port
 - `ssl.SSLCertVerificationError` - expired/unmatched cert or key files, or server s
 - `reason_code 0` - connection success
 - `1` - unacceptable protocol version, broker doesn't support our mqtt ver
 - `2` - identifier rejected, the `client_id` is not acceptable to the broker
 - `3` - server unavailable, refusing connection
 - `4` - bad username/password, invalid or wrong credentials
 - `5` - not authorized, not allowed
 - `7` - alternative authentication required, wants a different auth method
 - `8` - flow control, sending the messages too fast
 - `9` - quota exceeded, too many connections
 - `14` - server error, internal error
 - `MQTT_ERR_NO_CONN` / `MQTT_ERR_PROTOCOL` - protocol errors on the wire
 - `MQTT_ERR_SUCCESS` - the success return code for sub/pub
 - `UnicodeDecodeError` - payload is not utf-8, fall back to `repr(message.payload)`
- enabling the paho logger (`enable_logger`) shows the packets in the terminal, drop to `DEBUG` to watch the raw traffic

key functions used throughout (w.r.t the two files)

the broker/openssl side

- `net start/stop mosquitto` - the service control
- `openssl genrsa / req / x509` - key creation, CSR/cert signing, the whole chain

- mosquitto_sub / mosquitto_pub - terminal testing with --cafile/--cert/--key

the paho side

- mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="P1") - the client object, v2 callbacks asked for explicitly

- enable_logger() - pipes paho's own logs into our logging

- tls_set(ca_certs=, certfile=, keyfile=) - the client side of TLS in one setter

- username_pw_set("user", "userpass") - credentials without the terminal cmd dance

- connect(host, port) - opens the connection to the broker

- subscribe(topic) - returns (res, mid), checked against MQTT_ERR_SUCCESS

- publish(topic, msg) - returns an object with .rc, checked the same way

- loop_forever() - the blocking background loop with auto reconnect

- is_connected() - checked before disconnect() so we never disconnect twice

- disconnect() - clean goodbye from the broker

the logging side

- logging.getLogger(**name**) - the named logger per module, the name shows in the terminal

- logging.basicConfig(level, format) - the one-time setup in **main**

- logger.exception(...) - logs the message WITH the full traceback, used in every except block

- [logger.info\(...\)](#) - the normal flow messages

the message handling side

- message.payload.decode("utf-8") - bytes to string, with the UnicodeDecodeError fallback

- message.topic - which topic the msg came in on

where it leads - Machine_Sensor_API

- the capstone of this folder, a Flask + PostgreSQL API for machines and sensor readings with bulk import of mqtt snapshot pickles, it uses the same pub/sub thinking, honest status codes and tls habits end to end, code lives in Machine_Sensor_API/

not used yet - next to learn

- QoS levels in practice - 0 at most once, 1 at least once, 2 exactly once, my de
- retained messages - the broker keeps the last msg on a topic for new subscriber
- last will and testament - the broker announces a client that died unexpectedly
- persistent sessions - queueing msgs for a disconnected subscriber
- mqtt 5 properties/features - the properties parameter my callbacks already rece